Introduction to the Internet

What is the Internet?

The Internet is ubiquitous as an infrastructure that transfers data between devices around the world. When we think of the Internet, it is worth distinguishing between the Internet's *infrastructure* and *applications* that use this infrastructure. The Internet's infrastructure consists of hardware and software components (e.g., optical links, routers, communication protocols, network naming services) that collectively implement the transfer of data between devices. Internet applications then build on top of this infrastructure's data delivery, with the World Wide Web as perhaps the Internet's best known application. I.e., you can think of the web as applications built on top of the Internet infrastructure (e.g. Facebook, Twitter) that you can access through a web browser (e.g. Firefox, Chrome). Other applications besides the web can use Internet infrastructure, too. Examples of non-web applications are Zoom or online games, or even Internet-of-things (IoT) devices like a sensor in your refrigerator or car.

In this class, we'll be focusing on the Internet's infrastructure – its software and hardware components and how they come together to build a global data delivery system.

Why is Internet Infrastructure Interesting?

As a system, the Internet has certain unique characteristics that makes designing its infrastructure an interesting challenge.

The Internet is designed for generality and to accommodate heterogeneity.

Another distinction worth drawing is between a specific networking technology vs. the Internet's approach to networking. There are many examples of specific networking technologies: e.g., Ethernet, WiFi, 5G/-cellular, optical networking, and so forth. Each of these offers a solution for how devices that implement the specific technology in question can exchange data: i.e., only devices that implement Ethernet can communicate in an Ethernet network, and the same is true for devices in a cellular network. By contrast, the Internet's infrastructure was designed to incorporate *any* networking technology and to ensure that any two devices on the Internet can exchange data regardless of the specific link or network technology by which they might connect to the Internet. Thus the Internet is not a new type of network technology (e.g. wireless communication existed before the Internet), but is instead about a completely new problem of tying together multiple networks that might be based on different technologies. Moreover, these individual technologies are constantly evolving, which means we can't aim for a fixed target (e.g. capacity and demand is constantly increasing by orders of magnitude).

In other words, the Internet is designed for *generality* (i.e., to work with any network technology) and *heterogeneity* (i.e., the technologies in question might be very different and evolving). These are unique and often quite challenging goals for system designers since we are, in effect, designing for the future.

In this class, we'll study some relevant technologies (e.g., Ethernet and cellular) but our main focus is on the Internet infrastructure in its entirety and how it incorporates disparate technologies.

The Internet is Federated

The Internet is a *federated* system: it consists of multiple independently owned and operated networks that are interconnected to carry data between users. This requires interoperability between operators. In other words, each network operator acts independently, but every operator has to cooperate in order to connect the entire world. Thus *all* network operators agree on some common protocol(s) in order to achieve global connectivity.

Federation enables the tremendous scale of the Internet. Instead of a single operator managing billions of users, we only need to focus on interconnecting all the different operators. Federation also allows us to build the Internet out of a huge diversity of technologies (e.g. wireless, optical), with a huge range of capabilities (e.g. home links with tiny capacity, or undersea cables with huge capacity).

However, federation also introduces several challenges. Competing entities (e.g., rival ISPs) cooperate in providing service, yet must do so in a manner that protects each entity's confidential information. When designing protocols, we have to consider real-life business incentives in addition to technical considerations.

Federation also complicates innovation. In other fields, companies can innovate by developing a new feature that no one else has. But on the Internet, if one network has a feature that no other network has, then this feature will not be available to the general Internet user. All networks on the Internet must "speak a common language" (protocol) and any upgrades to the Internet have to be made with interoperability in mind.

The Internet is Scalable

As mentioned above, federation enabled the Internet to scale rapidly and in a decentralized manner. The massive scale of the Internet also means that any system we design has to support the massive range of users and applications on the Internet (e.g. some need more capacity than others, some may be malicious).

The worldwide scale of the Internet means that our systems and protocols need to operate asynchronously. Data can't move faster than the speed of light (and often moves much slower than that). Suppose you send a message to a server on the other side of the world. By the time your message arrives, your CPU might have executed millions of additional instructions, and the message you sent might already be outdated.

The scale of the Internet means that even sending a single message can require interacting with many components (e.g. software, switches, links). Any of the components could fail, and we might not even know if they fail. If something does fail, it could take a long time to hear the bad news. The Internet was the first system that had to be designed for failure at scale. Many of these ideas have since been adopted in other fields.

In summary, designing the Internet required developing solutions that were general, embraced heterogeneity, enabled federation, and scaled to unprecedented levels. Solving this problem required a new design paradigm and established networking as a (relatively new) field within computer science. The design principles that underlie the Internet are now widely embraced in building modern, scalable software services.

Thus in studying the Internet, we will consider new challenges that are different from many traditional computer science fields. For example, unlike theory fields, we will not model the Internet using formal

models; unlike computer architecture or hardware fields, we will not define rigorous workloads or benchmarks that quantify the performance of the Internet. The Internet is simply too complex for traditional formal techniques, and too vast and diverse to measure its performance.

Moreover, unlike previous classes you might have taken, it's no longer enough to write code that runs on your local machine and computes the correct output. Instead, you must also make sure that you have *designed* your system to work under general and heterogeneous conditions. For example, you might have to consider questions such as: will your system scale to a billion users? will it work with some new networking technology that emerges in the future? does it align with the business relationships of different operators (otherwise, they might not agree to run your code)? will it work on heterogeneous devices such as a smartphone or a heavy-duty server? And so forth.

In short, learning how the Internet works will teach us how to *architect* a complex distributed system: e.g. reasoning about goals, constraints, and trade-offs in the design. Network architecture is more about thinking about designs, and less about proving theorems or writing code. It's more about considering tradeoffs, and less about meeting specific performance benchmarks. It's more about designing systems that are practical, and less about finding the optimal design. The Internet is not optimal, but has successfully balanced a wide range of goals.

Protocols

In this class, much of our focus will be on **protocols** that specify how entities exchange in communication. What is the format of the messages they exchange, and how do they respond to those messages?

For example, imagine you're writing an application that needs to send and receive data over the Internet. The code at the sender machine and the code at the recipient machine need to both agree on how the data is formatted, and what they should do in response to different messages.

Here's an example of a protocol. Alice and Bob both say hello, then Alice requests a file, and Bob replies with the file. To define this protocol, we need to define syntax (e.g. how to write "give me this file" in 1s and 0s), and semantics (e.g. Alice must receive a hello from Bob before requesting a file).

Different protocols are designed for different needs. For example, if Alice needs to get the file as quickly as possible, we might design a protocol without the initial hello messages. Designing a good protocol can be harder than it seems! We might also need to account for edge cases, bugs, and malicious behavior. For example, what if Alice requests a file, and Bob replies with hello? How should Alice respond?

Throughout this class, we'll see many protocols that have been standardized across the Internet. You'll sometimes see the acronym RFC (Request For Comments) when we mention a protocol. Many standards are published as RFC documents that are eventually widely accepted, though not all RFCs end up adopted. RFC documents are numbered, and sometimes protocols are referred to by their RFC number. For example, "RFC 1918 addresses" refers to addresses defined by that particular document.

There are different standards bodies responsible for standardizing protocols. The IEEE focuses on the lower-layer electrical engineering side. The IETF focuses on the Internet and is responsible for RFCs.

Components of the Internet's Infrastructure

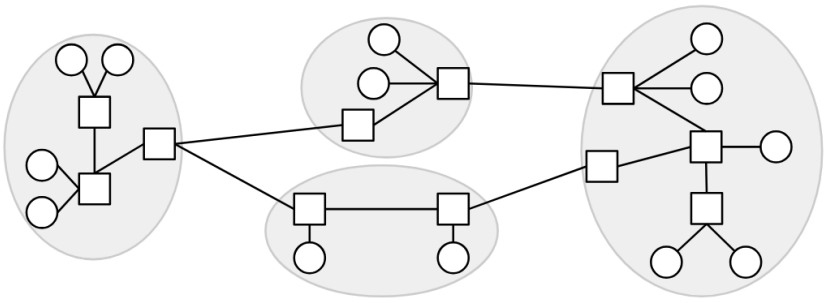
In the Internet, **end hosts** are machines (e.g. servers, laptops, phones) that send and receive data over the Internet. If two machines want to communicate directly, we might build a **link** between them. That link could be using any one of a variety of technologies: e.g., Ethernet, WiFi, optical transport, etc. A *point-to-point* link interconnects exactly two machines, while a *shared* link (a.k.a "multi access" link) may have multiple machines connected to the same physical link.

What happens as we increase the number of hosts that want to exchange data? One option might be to add a bunch of links between these hosts but, clearly, this would not scale very well! Instead, we use a **switch** (also called a **router**). A switch is a machine that isn't sending or receiving its own data, but instead exists to help the end hosts communicate with each other. A switch has multiple links that connect it to hosts and/or other switches. A switch can take the data it receives on one link and sends it out on a different outgoing link, an operation we call "forwarding" data. By transferring data across multiple links, switches enable hosts to communicate even if they are not directly connected by a link. Examples of switches are the router in your home, or larger routers deployed by Internet service providers (e.g. AT&T).

In these notes, we'll typically draw end hosts as circles, and routers as squares and depict point-to-point vs. shared links as shown in Figure TODO.

If we use links and switches to connect several nearby hosts (e.g. all the computers in Soda Hall), we get a **local area network (LAN)**. Typically, links within a single LAN are based on the same technology (e.g., Ethernet).

We can then connect different LANs to form a larger network. For example, we might add a new switch that has links connecting it to each of: our Ethernet LAN in Soda Hall, a different LAN in Cory Hall and our campus-wide WiFi network. Together, these might form UC Berkeley's network. Next, we might add a link connecting a switch in UC Berkeley's network to one in AT&T's network, thus connecting UC Berkeley to AT&T and (via AT&T) to any other network that AT&T is connected to. In this manner, We can connect individual networks to each other to form the Internet.



This brings us to an important point. The Internet is often described as a **network of networks**, as shown in Figure . There are lots of individual networks and what happens inside that individual network can be managed locally. For example, UC Berkeley's network infrastructure is owned and operated by our staff of *network operators*, while AT&T's infrastructure is independently managed by their own operational staff. Traditionally, commercial network operators such as AT&T are called **Internet Service Providers** (ISPs) though the term network **carriers** or **telcos** (for telecommunication providers) are also common.

The above picture shows the infrastructure of the Internet, but in this class, we'll also study the operators managing the infrastructure. Thus, in addition to the hardware and software infrastructure, we'll need to

think about these entities as real-life businesses and organizations, and consider their business and policy incentives. For example, if AT&T builds an undersea cable, they might charge a fee for other ISPs to send data through that cable.

Another question we'll need to answer is how to find paths across a network. When a switch receives data, how does it know where to forward the data, so that it gets closer to its final destination? This will be the focus of our routing unit.

We'll also need to make sure that there's enough capacity on these links to carry our data. This will be the focus of our congestion control unit.

Layers of the Internet

In this section, we'll build the Internet from the bottom-up, starting with basic building blocks and combining them to form the Internet infrastructure. We'll use the postal system as a running analogy, since it shares some characteristics with the Internet.

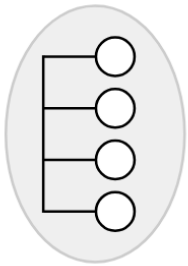
Layer 1: Physical Layer

First, we need some way to send a signal across space. In the postal system, this could be a mailman, the Pony Express, a truck, a carrier pigeon, etc.

In the Internet, we're looking for a way to signal bits (1s and 0s) across space. The technology could be voltages on an electrical wire, wireless radio waves, light pulses along optical fiber cables, among others. There are entire fields of electrical engineering dedicated to sending signals across space, but we won't go into detail in this class.

Layer 2: Link Layer

In the analogy, now that we have a way to send data across space, we can use that building block to connect up two homes. We could even try to connect up all the homes in the local town.

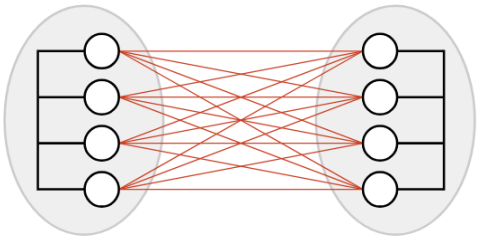


SR: Should we change the figure to not be a shared link?

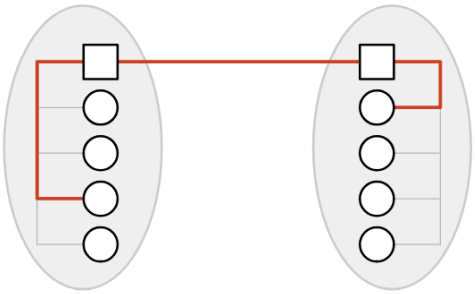
At Layer 2, we can also group bits into units of data called **packets** (sometimes called frames at this layer), and define where a packet starts and ends in the physical signal. We can also handle problems like multiple people simultaneously using the same wire to send data.

Layer 3: Internet Layer

We now have a way to connect everybody in a local area, but what if two people in different areas wanted to communicate? One possible approach is to add a bunch of links between different local networks, but this doesn't seem very efficient. (What if the two local networks were in different continents?)

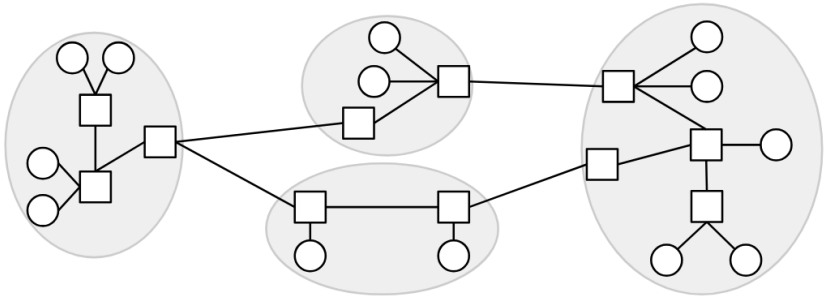


Instead, a smarter approach would be to introduce a post office in each network, and just connect the two post offices together. Now, if someone in network A wants to communicate with someone in network B, they can send the mail to the post office in network A. This post office forwards the mail to the post office in network B, which delivers the mail to the destination.



In the Internet, the post office receiving and redirecting mail is called a **switch** or **router**.

If we build additional links between switches, we can connect up local networks. With enough links and local networks, we can connect everybody in the world, resulting in the Internet.



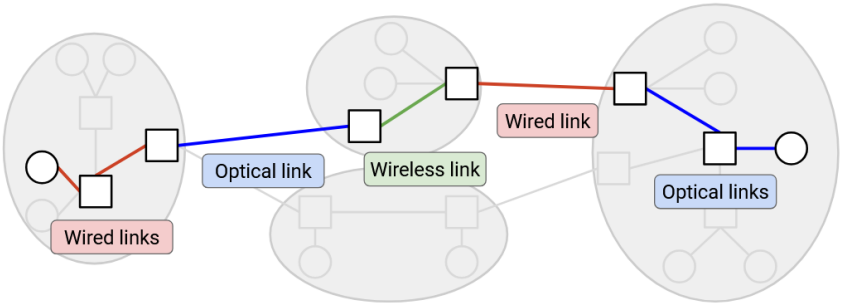
One question we'll need to answer is how to find paths across a network. When a switch receives a packet, how does it know where to forward the packet, so that it gets closer to its final destination? This will be the focus of our routing unit.

We'll also need to make sure that there's enough capacity on these links to carry our data. This will be the focus of our congestion control unit.

This picture now shows the infrastructure of the Internet, but in this class, we'll also study the operators managing the infrastructure. In the analogy, these are the people who build and manage the post office. On the Internet, the operators are **Internet service providers** like AT&T, Amazon Web Services, or even UC

Berkeley, who own and operate Internet structure. In addition to the hardware and software infrastructure, we'll need to think about these entities as real-life businesses and organizations, and consider their economic and political incentives. For example, if AT&T builds an undersea cable, they might charge a fee for other ISPs to send data through that cable.

In the network, different links might be using different Layer 2 technology. Some links might use wired Ethernet, and other links might use optical fiber or wireless cellular technology. At Layer 2, we figure out how to send a packet inside a local network, across the link(s) in that network, using the specific technology in that network. Then, at Layer 3, we use the ability to send packets along links as a building block to send packets anywhere in the Internet. As the packet hops across different networks, it may be transmitted across lots of different types of links.



In our analogy, we can see a distinction between homes and post offices. The homes are sending and receiving letters to each other. The post offices aren't sending or receiving their own mail, but they're helping to connect up the other homes.

Layers of Abstraction

As we've built up the Internet, you might have noticed that we've been decomposing the problem into smaller tasks and abstractions.

"Modularity based on abstraction is the way things are done." (Barbara Liskov, Turing lecture). This is how we build and maintain large computer systems. Modularity is especially important for the Internet because the Internet consists of many devices (hosts, routers) and many real-world entities (users, tech companies, ISPs), and having everybody agree on the breakdown of tasks is what enables the Internet to work at scale. SR: fix

One major advantage of this layered, network-of-network approaches is, each network can make its own decisions about how to move data. For example, as your packet travels across Internet hops, some links might use wireless technology, and other links might use wired technology. The lower-layer protocols can change across different hops, and the Layer 3 protocol still works.

Layering also allows innovation to proceed in parallel. Different communities (e.g. hardware chip designers, software developers) can pursue innovation at different layers.

Layer 3: Best-Effort Service Model

It seems like we've built something that can send data anywhere in the world, so why not stop here? There are two issues with Layer 3 that we still need to solve.

The first issue involves the Layer 3 service model. If you use the Layer 3 infrastructure to send messages across the Internet, what service model does the network offer to you as a user? You can think of the service model as a contract between the network and users, describing what the network does and doesn't support.

Examples of practical SR: fix service models might include: The network guarantees that data is delivered. Or, the network guarantees that data is delivered within some time limit. Or, the network doesn't guarantee delivery, but promises to report an error on failure.

The designers of the Internet didn't support any of those models. Instead, the Internet only supports **best effort** delivery of data. If you send data over Layer 3, the Internet will try its best to deliver it, but there is no guarantee that the data will be delivered. The Internet also won't tell you whether or not the delivery succeeded.

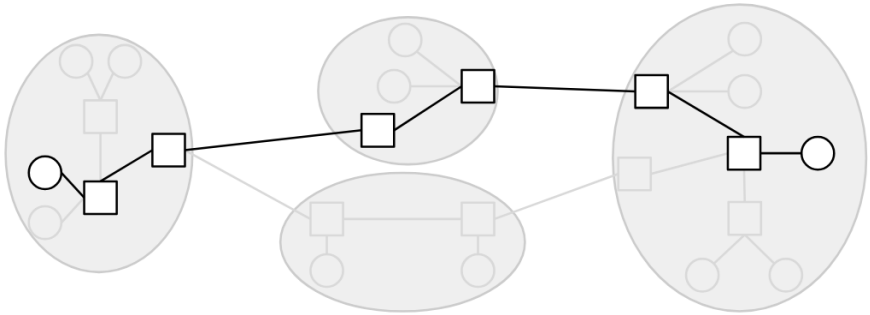
Why did the designers choose such a weak service model? One major reason is, it is much easier to build networks that satisfy these weaker demands. SR: fix

Layer 3: Packets Abstraction

So far, up to Layer 3, we've been thinking about sending each message through the Internet independently. More formally, the primary unit of data transfer at Layer 3 is a **packet**, which is some small chunk of bytes that travel through the Internet, bouncing between routers, as a single unit.

The second issue at Layer 3 is: Packets are limited in size. If the application has some large data to send (e.g. a video), we need to somehow split up that data into packets, and send each packet through the network independently.

With this packet abstraction, we can now look at the life of a packet as it travels across the network. The sender breaks up data into individual packets. The packet travels along a link and arrives at a switch. The switch forwards the packet either to the destination, or to another switch that's closer to the destination. The packet hops between one or more switches, each one forwarding the packet closer, until it eventually reaches its destination. Note that because of the best-effort model, any of the switches might drop the packet, and there's no guarantee the packet actually reaches the destination.



Layer 4: Transport

We've identified two issues at Layer 3 so far. Large data has to be split into packets, and IP is only best-effort.

To solve both of these problems, we'll introduce a new layer, the **transport layer**. This layer uses Layer 3 as a building block, and implements an additional protocol for re-sending lost packets, splitting data into packets, and reordering packets that arrive out-of-order (among other features).

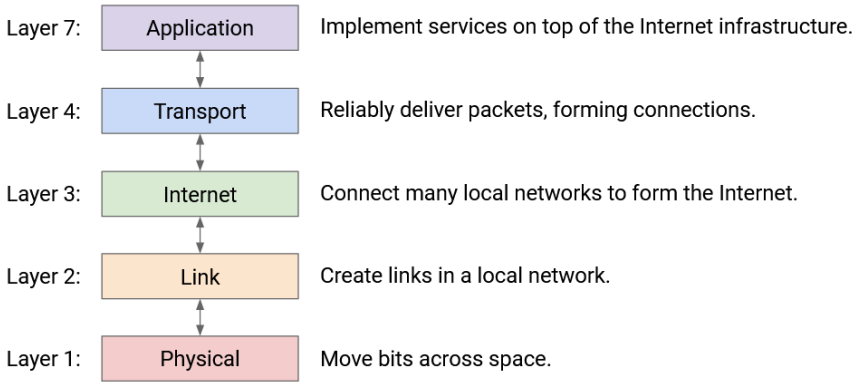
The transport layer protocol allows us to stop thinking in terms of packets, and start thinking in terms of **flows**, streams of packets that are exchanged between two endpoints. SR: fix

Layer 7: Application

The application layer being built on top of the Internet is a powerful design choice. SR: fix If, at lower layers, we built infrastructure for specifically transferring videos between end hosts, then email clients would have to build their own separate infrastructure for transferring emails. The Internet's design allows it to be a general-purpose communication network for any type of application data.

In this class, we'll focus more on the infrastructure supporting the application layer (e.g. the mailman, the post offices) and less on the applications themselves (e.g. the contents of the mail). We'll see some common application protocols near the end of the class, though.

Now that we've seen all the layers, notice that each layer relies on services from the layer directly below, and provides services to the layer directly above. For example, someone writing a Layer 7 (application) protocol can assume that they have reliable data delivery from Layer 4. They don't have to worry about individual packets being lost, since that's what Layer 4 already dealt with.



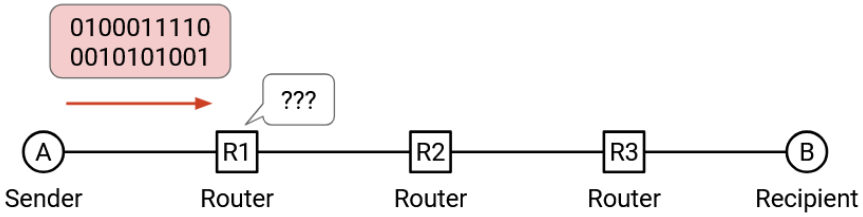
Two layers interact directly through the interface between them. There's no practical way to skip layers and build Layer 7 on top of Layer 3, for example.

Note: You might have noticed we skipped Layers 5 and 6. In the 1970s, when the layers were first standardized, the designers thought that these layers were needed, but they're obsolete in the modern Internet. If you're curious, the session layer (5) was supposed to assemble different flows into a session (e.g. loading various images and ads to form a webpage), and the presentation layer (6) was supposed to help the user visualize the data. Today, the functionality of these layers is mostly implemented in Layer 7.

Headers

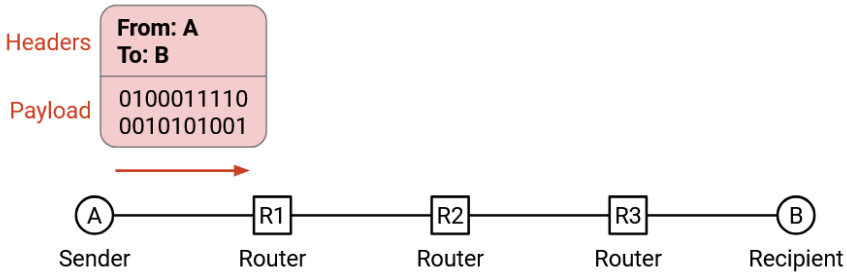
Why Do We Need Headers?

In the previous section, we saw that at Layer 3, data travels across the Internet in packets. Suppose an application wants to send a file over the Internet. We can take some bits of the image, put them in a packet, and send them over the Internet. When a switch receives this sequence of 1s and 0s, it has no idea what to do with these bits.



In the analogy, if I write a letter to my friend, and hand it to the post office, the post office has no idea what to do with it. Instead, we should put the letter inside an envelope, and write some information on the envelope (e.g. my friend's address) that tells the post office what to do with the letter.

Just like the envelope, when we send a packet, we need to attach additional metadata that tells the network infrastructure what to do with that packet. This additional metadata is called a **header**. The rest of the bits (e.g. the file being sent, the letter inside the envelope) is called the **payload**.



In the analogy, the post office shouldn't be reading the contents of my letter. It should only read what's on the envelope to decide how to send my letter. Similarly, the network infrastructure should only read the header to decide how to deliver the data.

The recipient cares about the inside of the letter, not the envelope. Similarly, the application at the end host cares about the payload, not the header. That said, the end hosts still need to know about headers, in order to add headers to packets before sending them.

Headers are Standardized

You can also think of headers as the API between the end hosts sending/receiving data, and the network infrastructure carrying the data. When we write software, we need to decide on the interface that users

will use to interact with our code (e.g. what functions users can call, the parameters to those functions). Similarly, the information in the header is how users access functions and pass parameters to the network.

Everybody on the Internet (every end host, every switch) needs to agree on the format of a header. If Microsoft Windows changes the code in its operating system to send packets with a different header structure, nobody else will understand the packets being sent.

This also means we need to be careful about designing headers. Once we design a header and deploy it on the Internet, it's very hard to change the design (we'd have to get everybody to agree to change it). This is why standards bodies can spend years designing and standardizing headers.

What Should a Header Contain?

What information should we put in the header?

The header should definitely contain the destination address, which tells us where to send the packet.

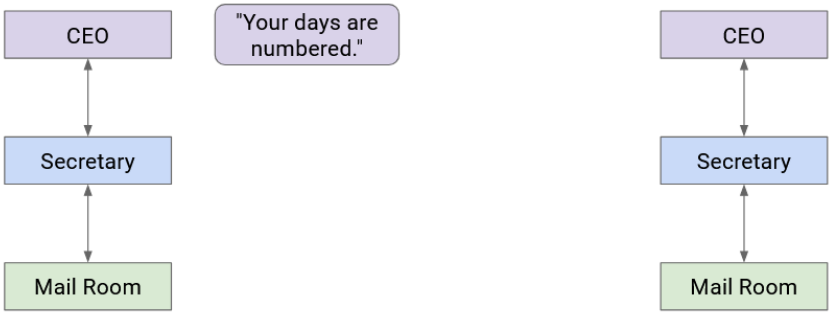
Headers could also contain other information that's not required, but is useful to have. Technically, the source address is not required to deliver the packet, but in practice, we almost always include the source address in the header. This allows the recipient to send replies back to the sender.

The header could also include a checksum, to ensure that packet is not corrupted while in transit.

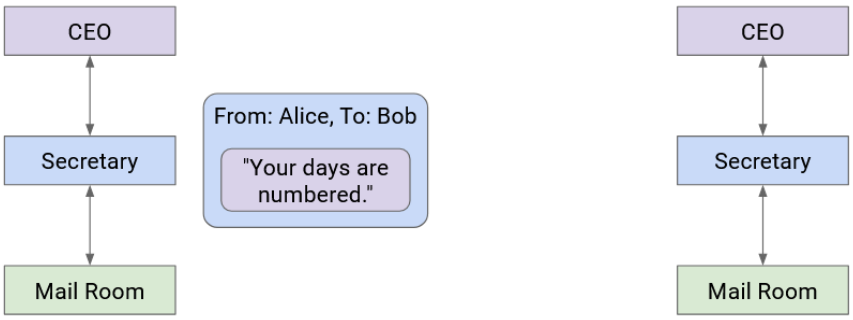
The header could also contain other metadata like the length of the packet. Note that packets can vary in size (e.g. the user might only need to send a few bytes).

Multiple Headers

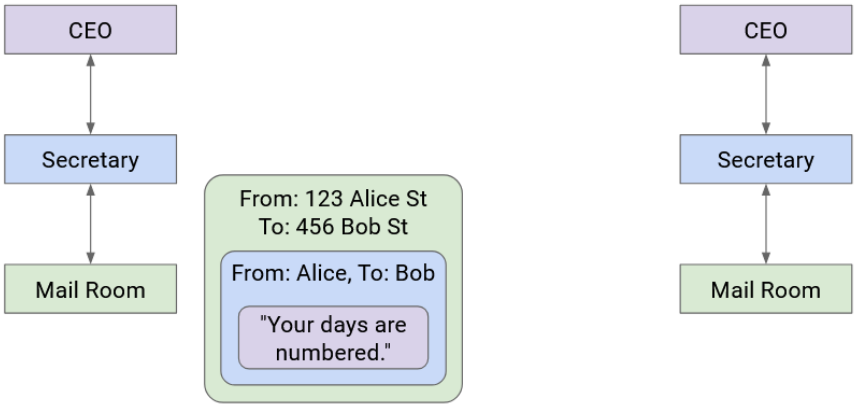
Let's go back to the postal analogy briefly. Suppose the boss of Company A wants to write a letter to the boss of Company B. How does the message get sent?



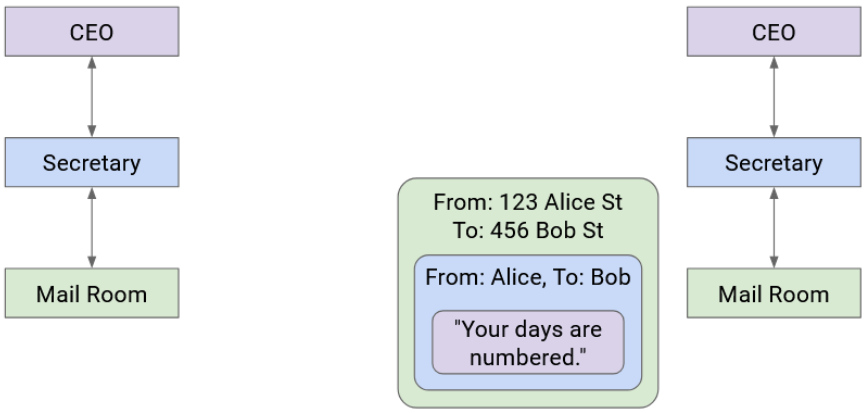
Company A's boss folds the letter and hands it to their secretary. Then, the secretary puts the letter in an envelope with Company B's boss's full name.



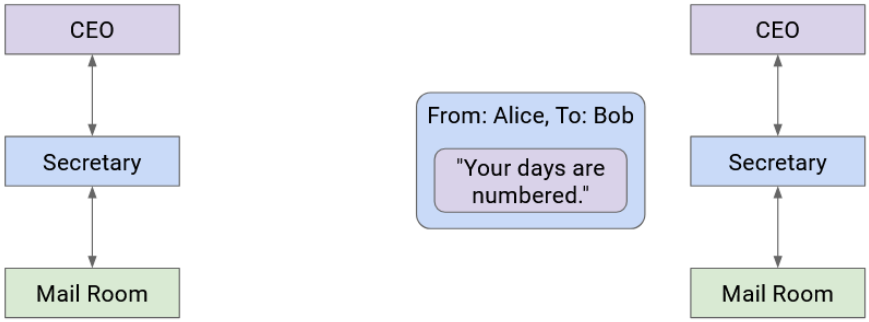
The secretary passes this letter to the mailroom. The postal worker puts the letter in a box with Company B's street address on it, and puts the package in a delivery truck.



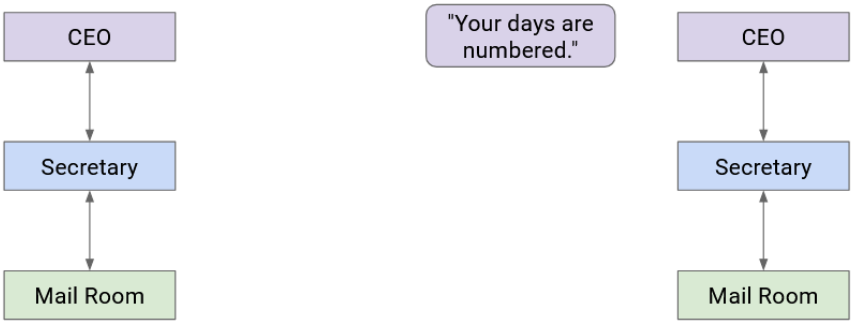
At this point, the letter itself is wrapped in multiple layers of identifying information (envelope, box). The delivery company sends the letter to Company B (possibly across several trucks, planes, mailmen, etc.).



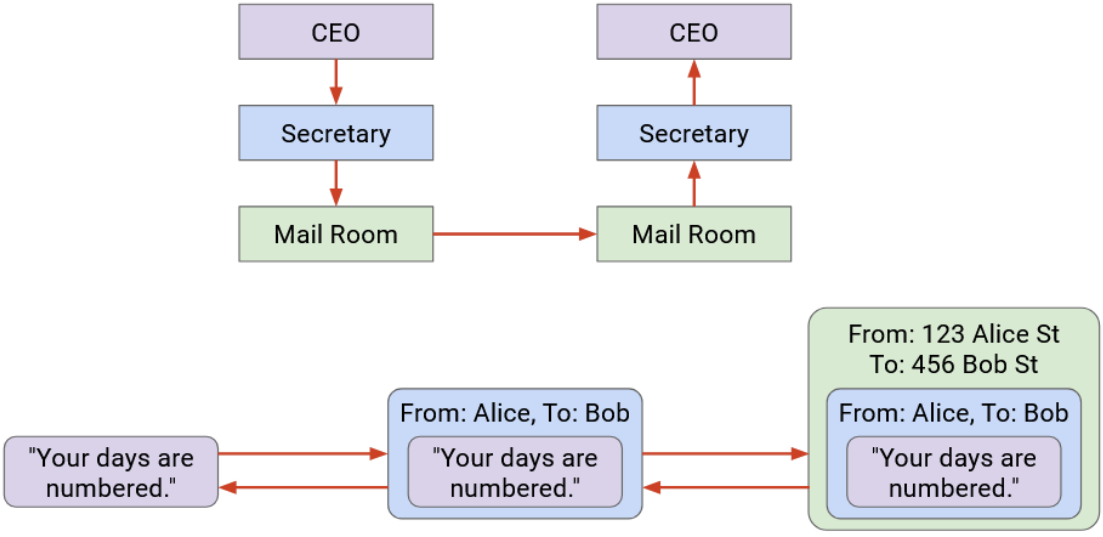
When the letter reaches Company B, the mailroom removes the box and passes the envelope to the secretary.



Then, the secretary sees the boss's name on the envelope, removes the envelope, and passes the letter up to the Company B boss.

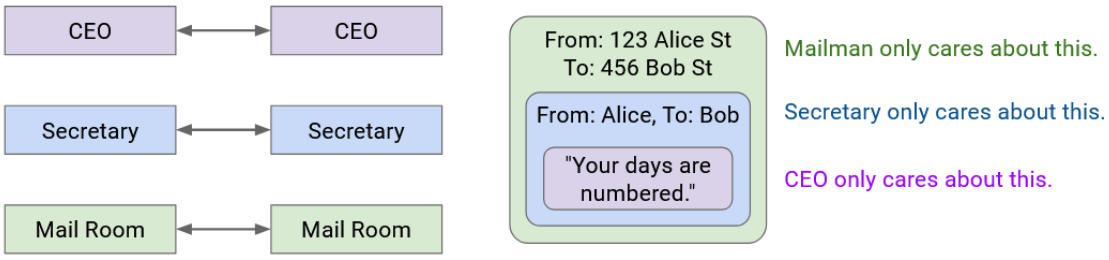


Notice that as we moved to lower abstraction layers, we wrapped more headers around the data. Then, as we moved to higher abstraction layers, we peeled layers off the data.



Each layer only has to understand its own header, and is "communicating" (in some sense) with its peers at the same layer. When Secretary A writes the name on the envelope, that's meant for Secretary B to read (not the mailmen, or the boss).

More formally, on the Internet, peers at the same layer communicate by establishing a protocol at that layer. The protocol only makes sense to entities at that specific layer.



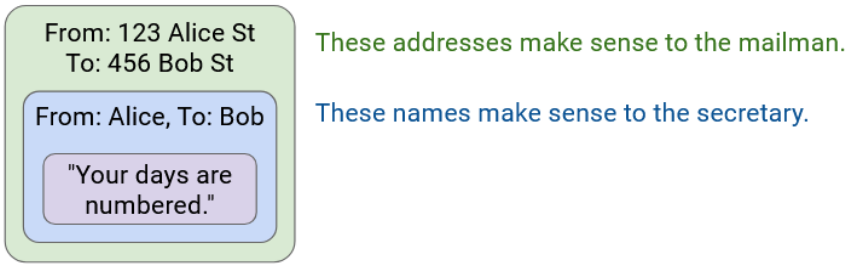
Note that some layers offer multiple choices of protocol (e.g. wireless or wired protocols at Layer 2). In these cases, the two people communicating need to use the same choice of protocol. A wired sender can't talk to a wireless recipient.

Addressing and Naming

Earlier, we said that our headers need to contain the address of the recipient. What actually is that address? Formally, a network address is some value that tells us where a host is located in the network.

As we look at the different layers in more detail, we'll see that different layers have different addressing schemes. If you want to send a letter inside Soda Hall, you could write the destination address as 413 Soda Hall, and the people in the building know where to deliver the letter. By contrast, if you want to send a letter to New York, you'd have to write a full street address like 123 Main Street, New York, NY.

Similarly, different layers in the Internet have different addressing schemes that work best for that particular layer. For example, sometimes a host is referred to by its human-readable name (e.g. www.google.com). Other times, that same host is referred to by a machine-readable IP address (e.g. 74.124.56.2), where this number somehow encodes something about the server's location (and could change if the server moves). Other times, that same host could be referred to by its hardware MAC address, which never changes. SR: fix

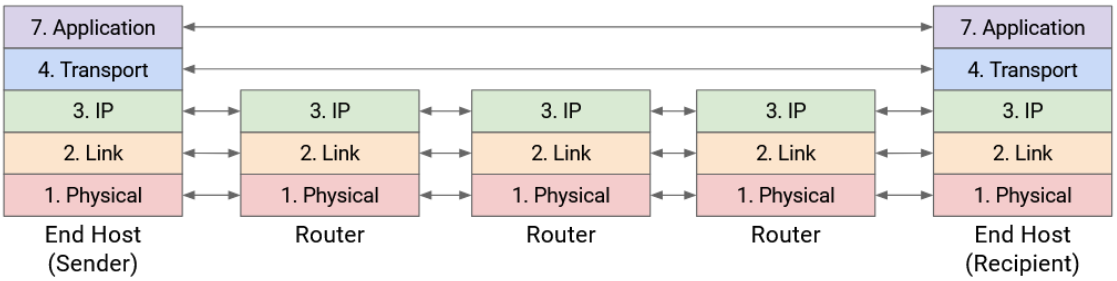


Layers at Hosts and Routers

The Internet is more than just a sender and a recipient. In addition to the two end hosts, there are routers forwarding the packet across multiple hops toward the destination. How do our ideas of layering and headers interact across all these machines?

The end hosts need to implement all the layers. Your computer needs to know about Layer 7 to run a web browser. Your computer also needs to know about Layer 1 to send the bits out along the wire. You'll also need all the layers in between in order for application-level data (the boss's letter) to be passed all the way down to the physical layer.

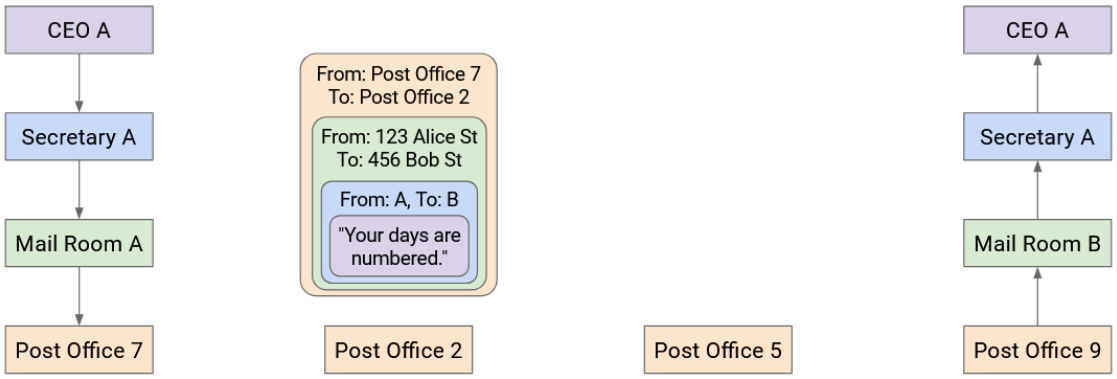
What about routers? The router does need Layer 1 for receiving bits on a wire, Layer 2 for sending packets along the wire, and Layer 3 for forwarding packets in the global network. However, the routers don't really need to think about Layer 4 and Layer 7. The router isn't running a web browser to display webpages, and the router doesn't need to think about reliability (recall, best-effort service model). SR: fix



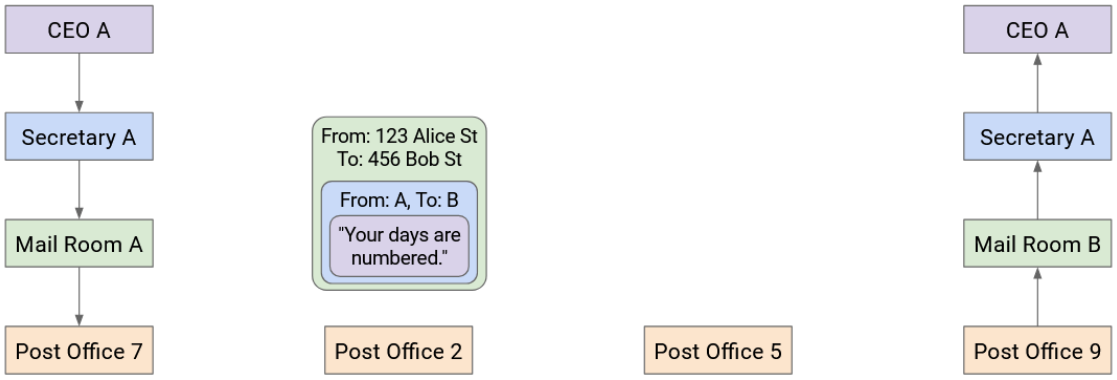
In summary: The lower 3 layers are implemented everywhere, but the top 2 layers are only implemented at the end hosts.

Multiple Headers at Hosts and Routers: Analogy

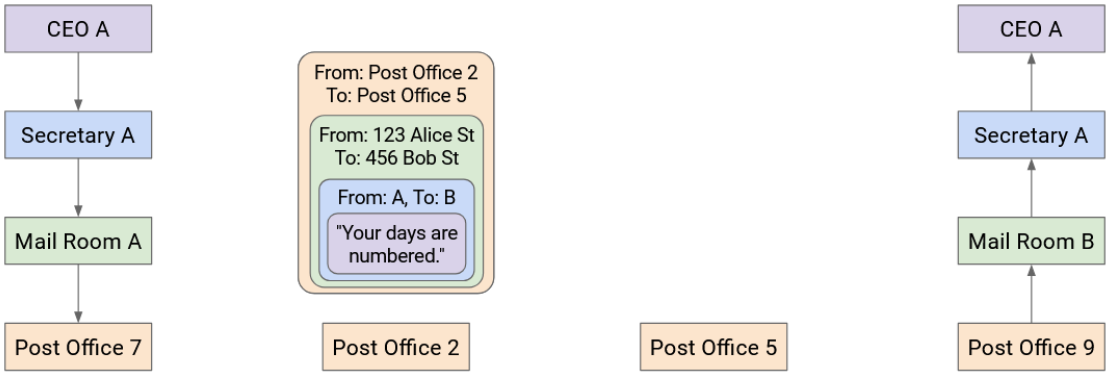
Let's think about sending mail again. Company A wrapped the letter in an envelope, which was then put in a box. The box doesn't magically travel to Company B. In fact, it might travel through several post offices.



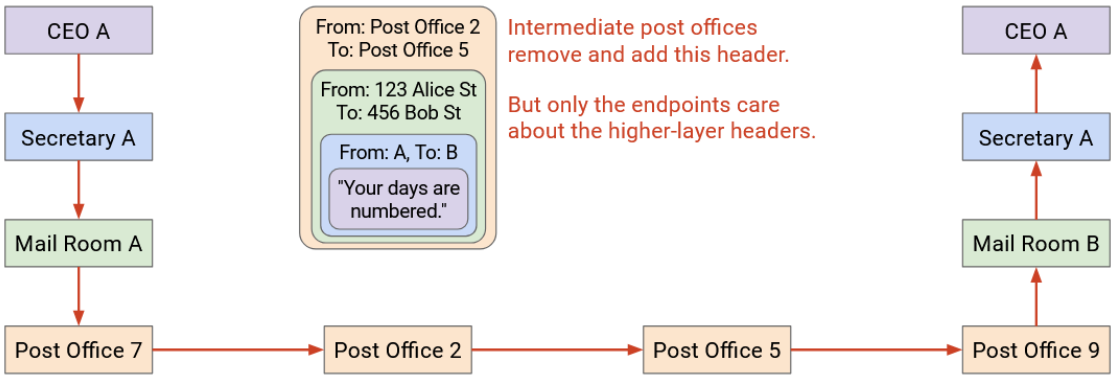
At each post office, the mailman opens the box and sorts through the mail. The mailman looks at the envelope (the next header revealed after opening the box), and sees that the envelope is meant for Company B.



The mailman then puts the envelope in another box, possibly different, so that the letter can reach the next post office on the way to Company B.



This process repeats at every post office. The box is opened, revealing the envelope inside. Then, the envelope goes in a new box, destined for the next post office. Notice that none of the post offices open the envelope to reveal the letter inside, because they don't need to read it.



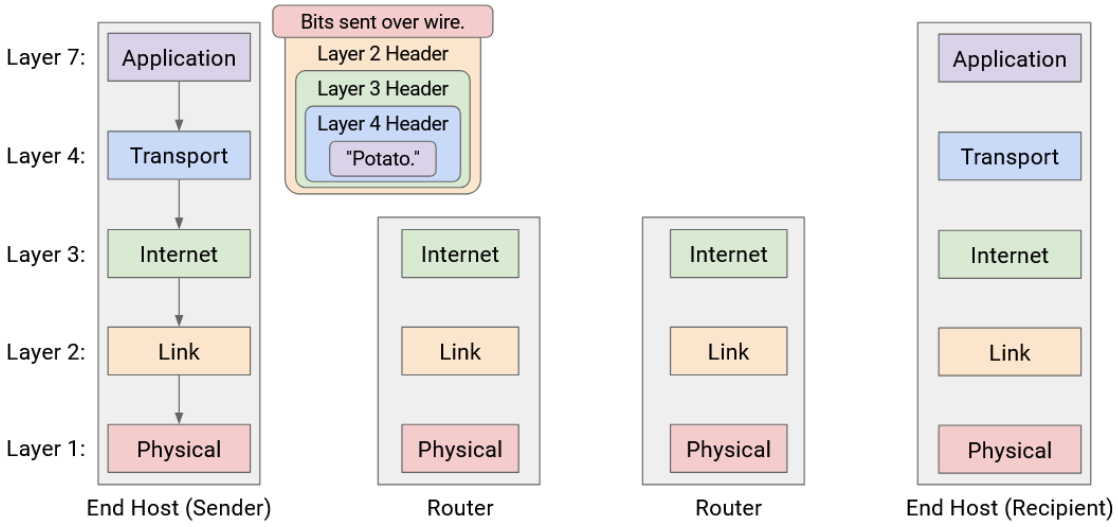
Eventually, the letter reaches Company B in a box, and this time, Company B opens the box, and the envelope, to reveal the letter inside.

Multiple Headers at Hosts and Routers

Now that we have the full picture with hosts and routers, let's revisit the demo of wrapping and unwrapping headers, as the packet takes multiple hops across the network.

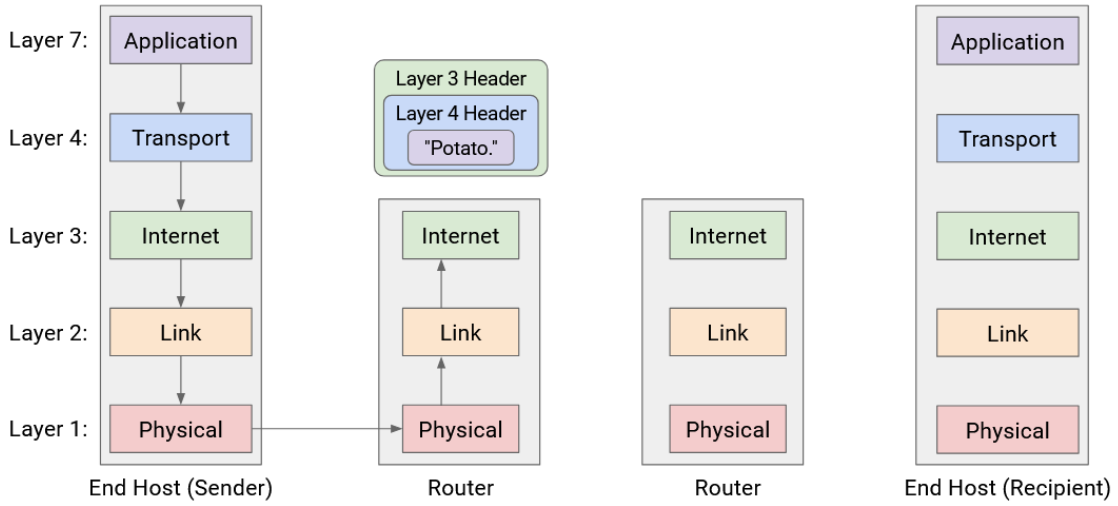
First, Host A takes the message and works its way down the stack, adding headers for Layer 7, 4, 3, 2, and 1. We now have a packet wrapped with headers for every layer.

The Layer 1 protocol sends the bits of this packet along the wire, to the first router on the way to the destination.

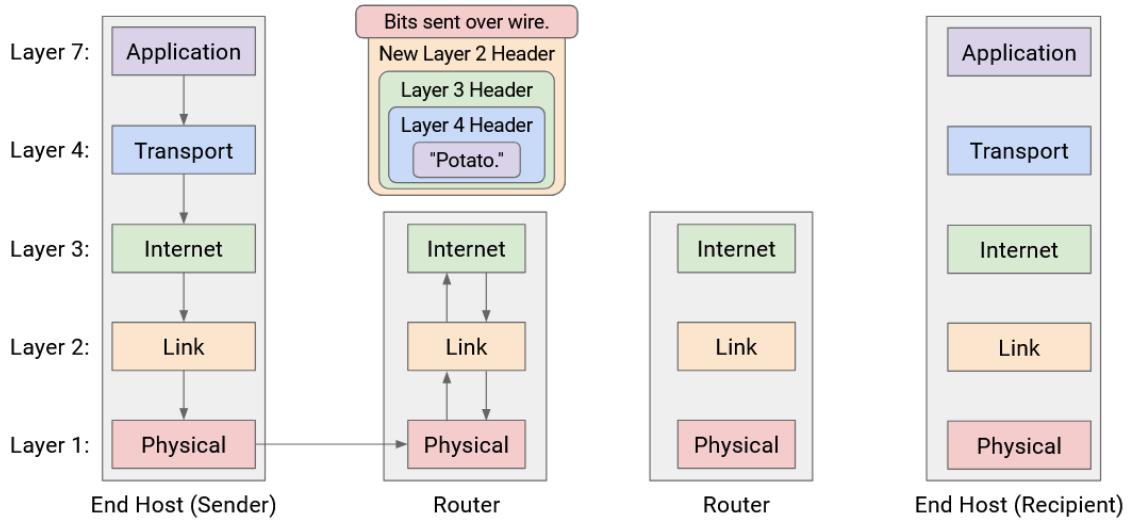


This router must forward the packet to the next hop, so that the packet eventually reaches Host B. We know that forwarding packets in the global network is a Layer 3 job. Therefore, the router must parse this packet up to Layer 3.

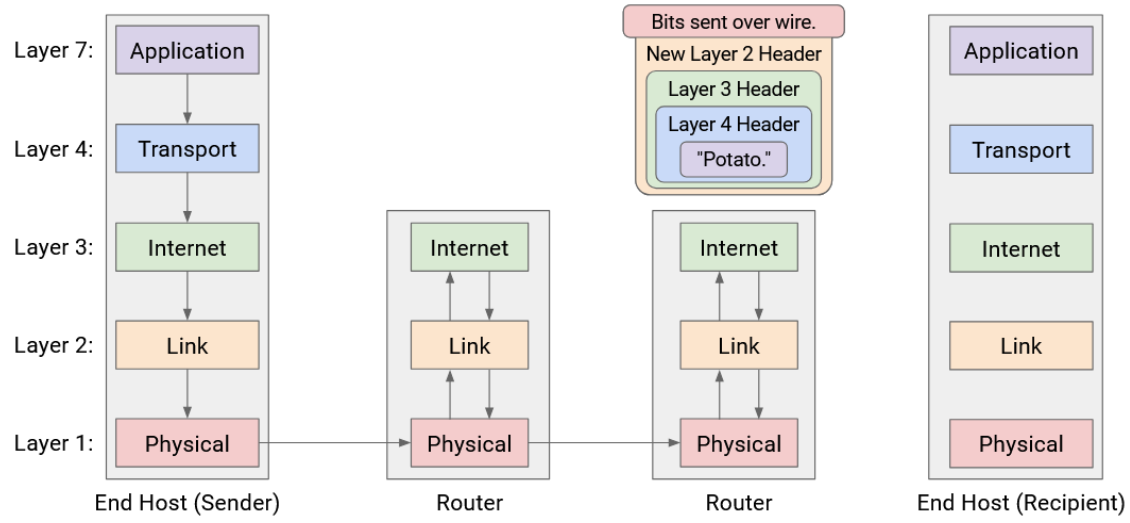
The router reads and unwraps the Layer 1 and Layer 2 headers, revealing the Layer 3 header underneath. The router reads this header to decide where to forward the packet next. SR: fix



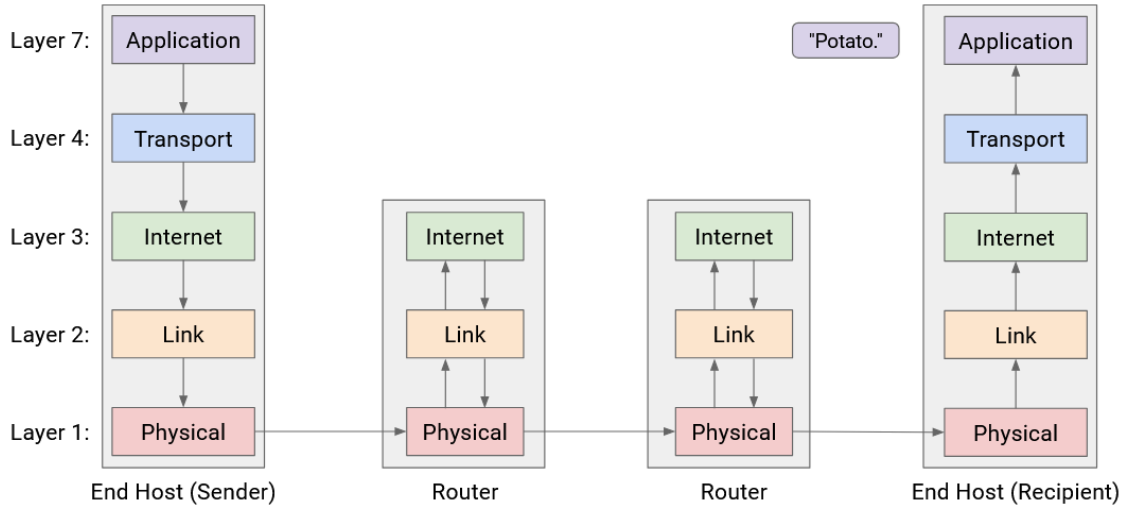
Now, to pass the packet along to the next hop, the router must go down the stack again, wrapping new Layer 2 and Layer 1 headers, and then sending the bits along the wire to the next hop.



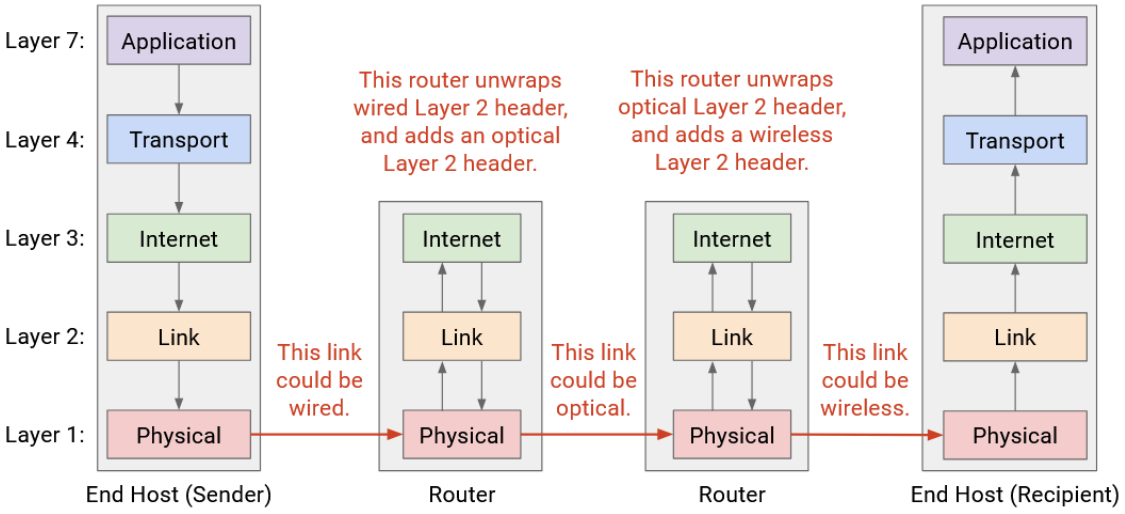
This pattern repeats at every router: Layers 1 and 2 are unwrapped to reveal the Layer 3 header, and then new Layer 2 and Layer 1 headers are wrapped before sending the packet to the next hop. Notice that none of the routers look beyond the Layer 3 protocol, because the upper layers are only parsed by the end hosts.



Eventually, the packet reaches Host B, who unwraps every layer, one by one: Layer 1, 2, 3, 4, 7. Host B has successfully received the message!

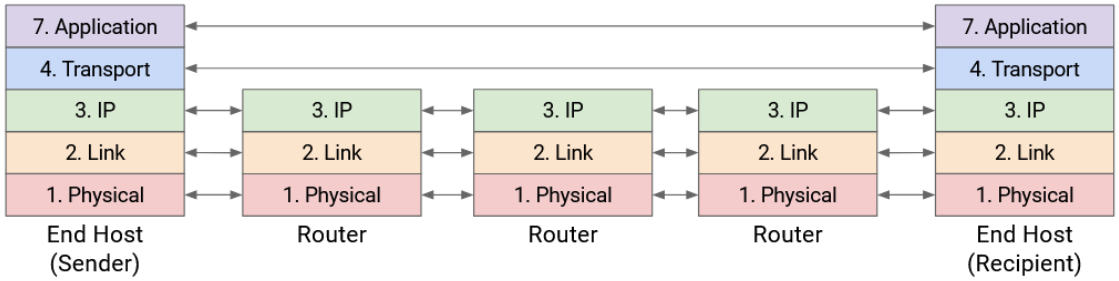


One consequence of this layering scheme is that each hop can use different protocols at Layer 2 and 1. For example, the first hop could get sent along a wire, and the initial Layer 2 and 1 headers used by Host A and the first router can be for a wired protocol. By contrast, a later hop could get sent along a wireless link, and the Layer 2 and 1 headers used by the routers on either end of that hop can be for a wireless protocol.



More generally, we said that each layer only needs to communicate with its peers at the same layer. We can now see this at play across all the layers. At Layers 4 and 7, the two hosts must speak the same protocols to send and receive packets. The host's peer is the other host. SR: fix

By contrast, at Layers 1 and 2, the router must speak the same protocol as the previous-hop and the next-hop router, so that the router can receive packets from the previous hop and send packets to the next hop. The router's peers are its neighboring routers along the path.



In summary: Each router parses Layers 1 and 2, while the end hosts parse Layers 1 through 7.